WEEK-3

1. Program to implement Banker's algorithm for a set of processes.

Aim: To implement Banker's algorithm for a set of processes to find the safe sequence of execution of programs.

Algorithm:

1. Start.
2. Read the number of processes n and the number of resource types m.
3. Read the Allocation Matrix, which represents the resources currently allocated to each process.
4. Read the Maximum Matrix, which represents the maximum resources each process may require.
5. Read the Available Resources vector.
6. Calculate the Need Matrix using:
7. Initialize all processes as unfinished by setting Finish[i] = 0.
8. Set count = 0 to keep track of the number of processes that can safely finish.
9. Find an unfinished process Pi whose Need is less than or equal to the Available resources for every resource type.
10. If such a process is found:
    - Assume Pi completes successfully.
    - Release its allocated resources:
    - Mark the process as finished:
      Finish[i] = 1
    - Store the process in the Safe Sequence.
    - Increase count by 1.
11. Repeat Steps 9–10 until all processes are finished or no suitable process can be found.
12. If count == n:
    - The system is in a SAFE state.
    - Display the Safe Sequence.
13. Otherwise:
    - The system is in an UNSAFE state.
    - Display that no safe sequence exists.
14. Display the Need Matrix and the result.
15. Stop.

Problem:

Find the safe sequence for the following processes with Aloocation and Maximum matrix ,Available resources provided also find the Need matrixfor the processes.

Process	Allocation matrix				Max matrix				Available/Work matrix			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	1	1	0	0	2	1	0	1	3	1	0
P1	1	4	4	1	1	6	5	2				
P2	1	3	6	5	2	3	6	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

Program:

```
#include <stdio.h>
 int main()
{
   int n, m;
   int alloc[10][10], max[10][10], need[10][10];
   int avail[10], finish[10] = {0}, safe[10];
   int i, j, count = 0;
   printf("============================================\n");
   printf("        BANKER'S ALGORITHM\n");
   printf("============================================\n");
   printf("Enter Number of Processes : ");
   scanf("%d", &n);
   printf("Enter Number of Resource Types : ");
   scanf("%d", &m);
   printf("\nEnter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
   {
      printf("P%d : ", i);
      for(j = 0; j < m; j++)
         scanf("%d", &alloc[i][j]);
   }
   printf("\nEnter Maximum Matrix:\n");
   for(i = 0; i < n; i++)
   {
      printf("P%d : ", i);
      for(j = 0; j < m; j++)
         scanf("%d", &max[i][j]);
   }
   printf("\nEnter Available Resources : ");
   for(j = 0; j < m; j++)
      scanf("%d", &avail[j]);
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         need[i][j] = max[i][j] - alloc[i][j];
      }
```

```
}
while(count < n)
{
   int found = 0;
   for(i = 0; i < n; i++)
   {
      if(finish[i] == 0)
      int flag = 1;
         for(j = 0; j < m; j++)
        {
            if(need[i][j] > avail[j])
            {
               flag = 0;
               break;
            }
         }
         if(flag)
         {
            for(j = 0; j < m; j++)
               avail[j] += alloc[i][j];
            safe[count++] = i;
            finish[i] = 1;
            found = 1;
         }
      }
   }
   if(found == 0)
      break;
}
printf("\n==========================================\n");
printf("Need Matrix\n");
printf("==========================================\n");
for(i = 0; i < n; i++)
{
   printf("P%d : ", i);
   for(j = 0; j < m; j++)
      printf("%d ", need[i][j]);
   printf("\n");
}
printf("\n==========================================\n");
if(count == n)
{
   printf("System is in SAFE State.\n");
```

```
        printf("Safe Sequence : ");
            for(i = 0; i < n; i++)
            {
                printf("P%d", safe[i]);
                if(i != n - 1)
                    printf(" -> ");
            }
            printf("\n");
        }
        else
        {
            printf("System is NOT in SAFE State.\n");
        }
        printf("=========================================\n");
        return 0;
}
```

OUTPUT

TESTCASE 1:

```
i071A6225\Bankers.exe
Enter Number of Processes : 5
Enter Number of Resource Types : 4

Enter Allocation Matrix:
P0 : 0 1 1 0
P1 : 1 4 4 1
P2 : 1 3 6 5
P3 : 0 6 3 2
P4 : 0 0 1
4

Enter Maximum Matrix:
P0 : 0 2 1 0
P1 : 1 6 5 2
P2 : 2 3 6 6
P3 : 0 6 5 2
P4 : 0 6 5 6

Enter Available Resources : 1 3 1 0

=========================================
Need Matrix
=========================================
P0 : 0 1 0 0
P1 : 0 2 1 1
P2 : 1 0 0 1
P3 : 0 0 2 0
P4 : 0 6 4 2

=========================================
System is in SAFE State.
Safe Sequence : P0 -> P3 -> P4 -> P1 -> P2
=========================================
```

TESTCASE 2:

```
071A6225\Bankers.exe
Enter Number of Processes : 4
Enter Number of Resource Types : 3

Enter Allocation Matrix:
P0 : 0 1 0
P1 : 2 0 0
P2 : 3 0 2
P3 : 2 1 1

Enter Maximum Matrix:
P0 : 7 5 3
P1 : 3 2 2
P2 : 9 0 2
P3 : 2 2 2

Enter Available Resources : 3 3 2

========================================
Need Matrix
========================================
P0 : 7 4 3
P1 : 1 2 2
P2 : 6 0 0
P3 : 0 1 1

========================================
System is in SAFE State.
Safe Sequence : P1 -> P3 -> P0 -> P2
========================================
```

TESTCASE 3:

```
071A6225\Bankers.exe
Enter Number of Processes : 5
Enter Number of Resource Types : 3

Enter Allocation Matrix:
P0 : 1 0 1
P1 : 0 2 0
P2 : 1 1 1
P3 : 2 0 1
P4 : 0 1 2

Enter Maximum Matrix:
P0 : 3 2 2
P1 : 2 3 2
P2 : 4 3 2
P3 : 3 2 2
P4 : 2 2 3

Enter Available Resources : 2 2 2

========================================
Need Matrix
========================================
P0 : 2 2 1
P1 : 2 1 2
P2 : 3 2 1
P3 : 1 2 1
P4 : 2 1 1

========================================
System is in SAFE State.
Safe Sequence : P0 -> P1 -> P2 -> P3 -> P4
========================================
```